

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical
Practical – V

Roll No.: T033	Name: Ashwin Tripathi
Class: TYIT	Batch:1
Date of Assignment: 21-07-2026	Date/Time of Submission:21-07-2026

Part A - Theory:

Part B - Program

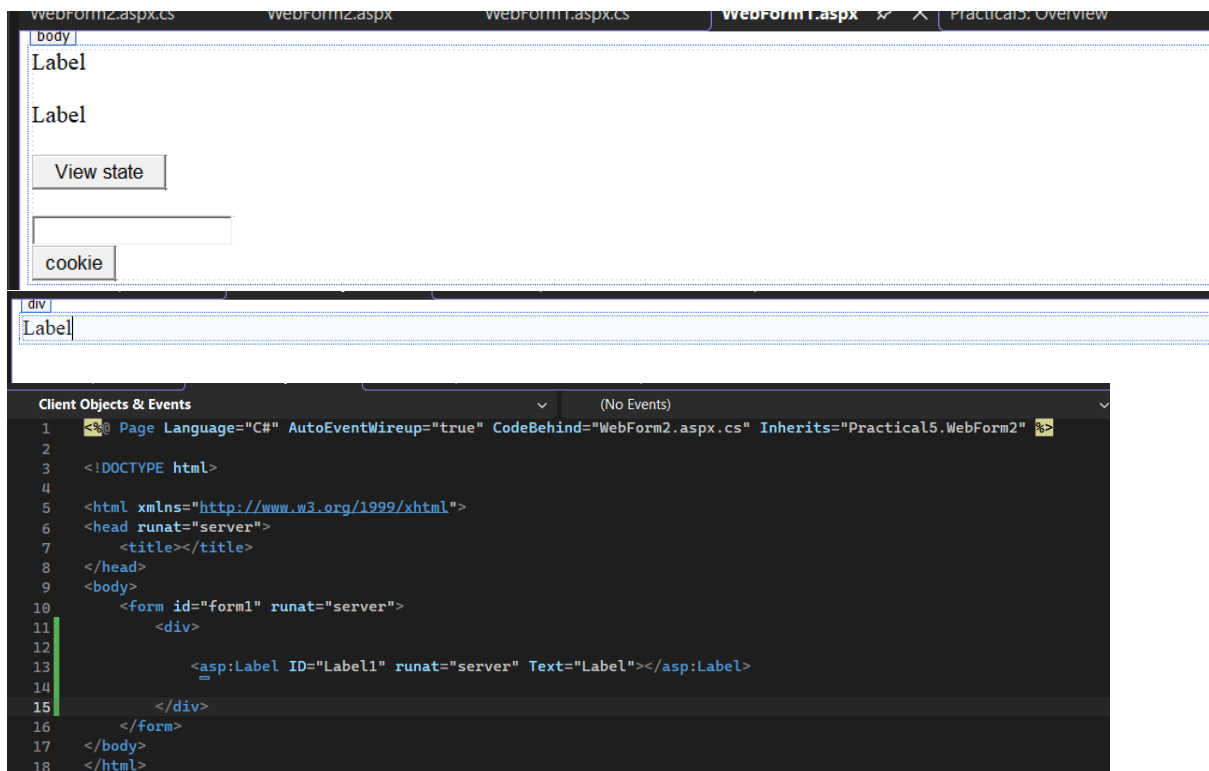
Aim :Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

1. Start.
2. Display labels, textbox, and buttons.
3. Click **View State** button.
4. Store/Retrieve data using ViewState.
5. Display ViewState value.
6. Enter text in the textbox.
7. Click **Cookie** button.
8. Store textbox value in a cookie.
9. Retrieve and display the cookie value.
- 10.Stop.

Code:

```
Client Objects & Events (No Events)
1 <%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebForm1.aspx.cs" Inherits="Practical5.WebForm1" %>
2
3 <!DOCTYPE html>
4
5 <html xmlns="http://www.w3.org/1999/xhtml">
6 <head runat="server">
7 <title></title>
8 </head>
9 <body>
10 <form id="form1" runat="server">
11 <div>
12
13 <asp:Label ID="Label1" runat="server" Text="Label"></asp:Label>
14 <br />
15 <br />
16 <asp:Label ID="Label2" runat="server" Text="Label"></asp:Label>
17 <br />
18 <br />
19 <asp:Button ID="Button1" runat="server" OnClick="Button1_Click" Text="View state" />
20 <br />
21 <br />
22 <asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>
23 <br />
24 <asp:Button ID="Button2" runat="server" OnClick="Button2_Click" Text="cookie" />
25
26 </div>
27 </form>
28 </body>
```



Output:

Label

View state = 2

View state

Ashwin

cookie

0

WelcomeAshwin

Label

Aim : Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm:

Code:

Step 1: Create Database

CREATE DATABASE ProductDB;

GO

USE ProductDB;

No need to create the table manually. EF Core will create it.

Step 2: Product Model (Product.cs)

```

public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Price { get; set; }
}

```

Step 3: AppDbContext (AppDbContext.cs)
using Microsoft.EntityFrameworkCore;

```

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
    : base(options)
    {
    }

    public DbSet<Product> Products { get; set; }
}

```

Step 4: appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection
      =True;TrustServerCertificate=True;"
  }
}

```

If you use SQL Server Express:

```

"Server=.\SQLEXPRESS;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=True;"

```

Step 5: Program.cs

```

using Microsoft.EntityFrameworkCore;

```

```

var builder = WebApplication.CreateBuilder(args);

```

```

builder.Services.AddControllersWithViews();

```

```

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

```

```
var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Step 6: HomeController
using Microsoft.AspNetCore.Mvc;

```
public class HomeController : Controller
{
    private readonly ApplicationDbContext _context;

    public HomeController(ApplicationDbContext context)
    {
        _context = context;
    }

    public IActionResult Index()
    {
        Product p = new Product();

        p.Name = "Laptop";
        p.Price = 50000;

        _context.Products.Add(p);
        _context.SaveChanges();

        return Content("Product Inserted Successfully");
    }
}
```

Step 7: Install Packages
Open Package Manager Console

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
```

Step 8: Migration

Add-Migration InitialCreate

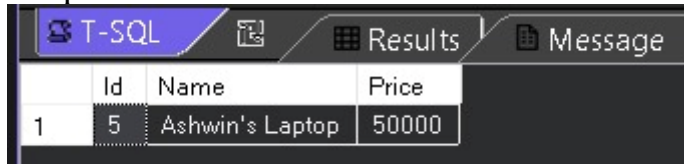
Update-Database

Step 09: Verify

Open SQL Server and run:

SELECT * FROM Products;

Output:



	Id	Name	Price
1	5	Ashwin's Laptop	50000

Aim : Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

Step 1: Add Employee.cs

namespace Practical5c.Models

```
{  
    public class Employee  
    {  
        public int Id { get; set; }  
        public required string Name { get; set; }  
        public required string department { get; set; }  
        public double salary { get; set; }  
    }  
}
```

Step 2 : Add AppDbContext.cs

using Microsoft.EntityFrameworkCore;

using Microsoft.Identity.Client;

using Practical5c.Models;

namespace Practical5c.Data

```
{  
    public class AppDbContext:DbContext  
    {  
        public AppDbContext(DbContextOptions<AppDbContext> options) :  
base(options) { }  
  
        public DbSet<Employee> Employees { get; set; }  
  
    }  
}
```

Step 3 : Add Program.cs

```
using Microsoft.EntityFrameworkCore;
using Practical5c.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));
```

Step 4 : Module -> Add -> new scaffolded Item

```
Practical5c EmployeesController

1
2 using Microsoft.AspNetCore.Mvc;
3 using Microsoft.EntityFrameworkCore;
4 using Practical5c.Models;
5 using Practical5c.Data;
6
7 public class EmployeesController : Controller
8 {
9     private readonly AppDbContext _context;
10 }
```

Step 5 : go to appsetting.json

```
"ConnectionStrings": {
  "MyConnection": "Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=true;TrustServerCertificate=true;",
  "AppDbContext": "Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true"
}
```

Output:

Create Employee

Name

department

salary

Create

[Back to List](#)

Index

[Create New](#)

Name	department	salary	
Ashwin	IT	120000	Edit Details Delete

Edit

Employee

Name

department

salary

[Back to List](#)

Delete

Are you sure you want to delete this?

Practical5c.Models.Employee

Name Ashwin

department IT

salary 120000

| [Back to List](#)

Aim : Authentication and Authorization in ASP.NET Core.

Algorithm:

1. Create an ASP.NET Core MVC project.
2. Enable Cookie Authentication in Program.cs.
3. Create Login action in HomeController.
4. Verify username and password.
5. Redirect to Secure page after successful login.
6. Use [Authorize] to protect the Secure page.
7. Logout and redirect to Login.

Code:

Program.cs

using Microsoft.AspNetCore.Authentication.Cookies;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

```

builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Home/Index";
    });

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

HomeController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;

namespace AuthDemo.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        public async Task<IActionResult> Index(string username, string password)
        {

```

```

        if (username == "admin" && password == "123")
        {
            var claims = new List<Claim>
            {
                new Claim(ClaimTypes.Name, username)
            };

            var identity = new ClaimsIdentity(claims,
                CookieAuthenticationDefaults.AuthenticationScheme);

            await HttpContext.SignInAsync(
                CookieAuthenticationDefaults.AuthenticationScheme,
                new ClaimsPrincipal(identity));

            return RedirectToAction("Secure");
        }

        ViewBag.Message = "Invalid Username or Password";
        return View();
    }

    [Authorize]
    public IActionResult Secure()
    {
        return Content("Welcome! Login Successful.");
    }

    public async Task<IActionResult> Logout()
    {
        await HttpContext.SignOutAsync(
            CookieAuthenticationDefaults.AuthenticationScheme);

        return RedirectToAction("Index");
    }
}

```

Views/Home/Index.cshtml

```
<h2>Login</h2>
```

```
<form method="post">
```

Username :

```
<input type="text" name="username" /><br /><br />
```

Password :

```
<input type="password" name="password" /><br /><br />
```

```
<input type="submit" value="Login" />
```

```
</form>
```

```
<p style="color:red">  
    @ViewBag.Message  
</p>
```

Output:

practical5d [Home](#) [Privacy](#)

Login

Username :

Password :

Login

Welcome Ashwin! Login Successful.